

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Ch.poorna teja	Reg. No.:	192472149
Section / Batch:	2024	Date:	22-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Code of Conduct

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO4 AT1-CODE CHALLENGE TASK

← Back

Language: Python C C++ Java

QUESTIONS

Q1
Activity Selection Problem
5 marks

1/1 answered

Q1 Activity Selection Problem

5 marks

A set of activities is given with their start and finish times, and only one activity can be performed at a time. The goal is to select the maximum number of non-overlapping activities so that overall utilization is maximized. Activities must be chosen based on an optimal strategy to avoid conflicts. Use a Greedy approach to determine the best selection. Input Format: n = number of activities start[] = start times finish[] = finish times Sample Input: n = 4 start = 1 3 0 5 finish = 2 4 6 7 Expected Output: Max Activities = 3

Your Code

```
1 n = int(input())
2
3 start = list(map(int, input().split()))
4 finish = list(map(int, input().split()))
5
6 # Combine start and finish times
7 activities = list(zip(start, finish))
8
9 # Sort by finish time
10 activities.sort(key=lambda x: x[1])
11
12 count = 0
13 last_finish = 0
14
15 for s, f in activities:
16     if s >= last_finish:
17         count += 1
18         last_finish = f
19
20 print("Max Activities =", count)
```

Submitted

Input

```
4
1 3 0 5
2 4 6 7
```

Output

Algorithm: Activity Selection Problem using Greedy Method

1. **Start**
2. Read the number of activities n.
3. Read the **start times** of all activities.
4. Read the **finish times** of all activities.
5. Combine each start time and finish time as an activity pair (start, finish).
6. **Sort all activities by their finish time** in ascending order.
7. Set count = 0 and last_finish = 0.
8. Traverse each activity (s, f):
 - If s >= last_finish:
 - Select the activity.
 - Increment count.
 - Update last_finish = f.
9. Print the maximum number of selected activities.
10. **Stop**